Introduction To Artificial Intelligence
AI-powered recommendation systems

LaTeX template adapted from:
European Conference on Artificial Intelligence

Anita Tabbali[1]

Other group members:
Alessia Lionetti[2], **Christopher Liu**[3], **Javeriah Zafar**[4]

Abstract. With growing number of movies available on online platforms, users often struggle to find content that truely matches their preferences. Recommendation systems help address this challenge by offering personalised suggestions and helping users find what suits them more quickly. In this project, a user-based collaborative filtering model was developed using the MovieLens dataset. The system applies KNN with cosine similarity and popular movies were filtered in to reduce sparsity before splitting the data into training and test sets. After evaluating the model using MAE, RMSE, Precision@20 and Recall@20, the optimised version showed clear improvements over a baseline model by providing more accurate and relevant recommendations. Overall, the system demonstrates a practical and effective approach to personalised movie recommendation, with opportunities for future enhancement through item-based KNN, factorisation methods, or hybrid models that incorporate additional movie information.

1 Introduction

Recommendation systems have become a key component in many online platforms, providing personalised suggestions and helping a significant number of individuals navigate large amounts of information. Whether they are suggesting products on e-commerce websites, recommending songs in music apps, or movies in movie platforms, these systems analyse user's decisions and preferences to suggest personalised content. As digital data continues to grow day by day, recommendation systems play a crucial role in reducing information overload and presenting the most relevant items to each user, ultimately improving the efficiency and user satisfaction. Enhancing the quality of recommendations increases the likelihood that users will remain active and renew their subscriptions [3].

Among these platforms, movie recommendation systems are particularly important. Services such as Netflix and Amazon prime video offer a vast library of movies and shows, which can make it difficult for users to decide what to watch. Therefore, building an effective movie recommendation system helps address this challenge. There are multiple ways to develop a recommendation system, which can be implemented using various algorithms. [10].
In this project, we develop a user-based k-nearest-neighbours(KNN) collaborative filtering model on the MovieLens 100k dataset [4], evaluated using rating error metrics and top-N recommendation metrics.

2 Background

Recommendation systems can be developed using a variety of approaches, including content-based filtering, collaborative filtering, and hybrid filtering.

Content-based filtering analyses item features and recommends items by matching them to those the user has liked in the past. Collaborative filtering, in contrast, focuses on user behaviour such as ratings or interactions, identifying patterns shared between users or items. Hybrid methods combine both approaches to overcome their individual limitations and improve recommendation quality. Collaborative filtering relies on users' historical preferences and rating behaviour to create recommendations. It compares rating patterns among users to identify those who share similar taste. User-based and item-based are the two common variations of collaborative filtering [5].

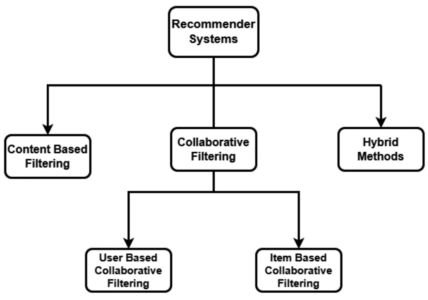
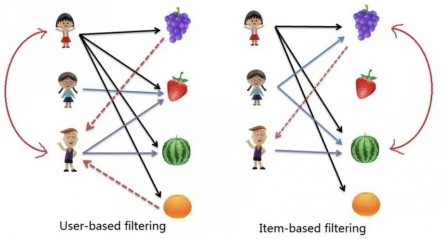


Figure 1. Collaborative recommendation system diagram

As shown in Figure 2 [7], user-based filtering looks for connections between users: if two users have rated many of the same movies in similar ways, the system considers them similar. In this regard, a movie liked by one user may be recommended to another if they have

[1] School of Computing and Mathematical Sciences, University of Greenwich, London SE10 9LS, UK
[2] School of Computing and Mathematical Sciences, University of Greenwich, London SE10 9LS, UK
[3] School of Computing and Mathematical Sciences, University of Greenwich, London SE10 9LS, UK
[4] School of Computing and Mathematical Sciences, University of Greenwich, London SE10 9LS, UK

not yet watched it. Item-based filtering takes a different approach by focusing on relationships between items. If a large number of users tend to rate two movies similarly, the system considers these movies to be similar and may recommend them together [2].



H

Figure 2. Difference between user-based and item-based CF

A crucial step in collaborative filtering is to measure similarity between users or items. Several similarity metrics can be used, but cosine similarity [8] is commonly applied to sparse datasets like MovieLens where most users have rated only a few movies. Cosine similarity measures the angle between two rating vectors; the smaller the angle, the more similar the vectors are. It is computed using the dot product of the vectors and normalising by their magnitudes [9]:

$$\cos(\vec{A}, \vec{B}) = \frac{\vec{A} \bullet \vec{B}}{\left\|\vec{A}\right\| * \left\|\vec{B}\right\|}$$

[5]

In user-based collaborative filtering, system uses an algorithm called k-nearest-neighbours (KNN), which identifies the most similar users to a target user by comparing their rating pattern. Each user is represented as a vector of movie ratings, and similarity is calculated using cosine similarity. The K most similar users become the target user's neighbours. The system then uses the neighbours' preferences to suggest new movies or estimate how the target user might rate an unseen movie [1].

To evaluate how well a recommendation system performs, several metrics can be used. Mean Absolute Value (MAE) and Root Mean Squared Error (RMSE), defined in equation 1 and 2, measure how close the model's predicted rating is to the actual rating. As shown in equation 3 and 4, Precision@K and Recall@K measure the quality of ranked recommendations generated for each user [6].

$$\text{RMSE} = \sqrt{\frac{\sum_{i=0}^{N-1}(y_i - \hat{y}_i)^2}{N}} \tag{1}$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{2}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{3}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{4}$$

3 Experiments and results

3.0.1 Environment and Dataset Setup

To build a KNN model, we used the MovieLens Latest dataset [4]. The files movies.csv and ratings.csv we used contains movieId, title, genres and userId, movieId, ratings, and timestamps respectively. Our data analysis shows that it contains 610 unique users and 9724 unique movies with an average rating of approximately 3.5.

3.0.2 Data Preprocessing

The ratings and movies were merged so each rating was linked to its movie title. And rows with missing titles were removed. Since collaborative filtering is sensitive to sparse data, popularity threshold was applied: the number of ratings were calculated for each movie and only those with at least 60 ratings were kept. This reduces noise in our system and improves the accuracy of recommendations. After filtering, our dataset was reduced to 41362 ratings over 450 movies.

3.0.3 Train–Test Split and Matrix Construction

The processed data was then split into training and test sets using an 80/20 split, with random state of 42 to ensure consistent results. A user–item matrix was then created from the training data using a pivot table where rows represent users, columns represent movies, and cells contain the user's rating. Missing ratings were replaced with zeros, producing a sparse matrix. This matrix was then converted to a Compressed Sparse Row (CSR) matrix to optimise memory usage and efficiency.

3.0.4 Model Construction (User-Based KNN)

A user-based KNN model was implemented using cosine similarity and a brute-force search algorithm. Cosine similarity was selected because it handles sparse rating data effectively and identifies similar user rating patterns. After fitting the model, two functions were used: one to predict a user's rating for a movie based on their k nearest neighbours, and another to generate a top-N recommendation list by aggregating neighbour preferences.

3.0.5 Baseline Model and Hyperparameter Tuning

To compare the optimised KNN model fairly, a simple baseline version of the same algorithm was first defined. This baseline uses minimal hyperparameter settings, including a very small number of neighbours, which generally leads to less accurate predictions. It serves as a reference point to show how performance changes when more appropriate hyperparameters are applied. After establishing the baseline, several parameters were adjusted to improve the model. In particular, the value of k was increased, allowing the system to rely on a larger and more reliable neighbourhood when estimating ratings. Additionally, the popularity threshold was raised to filter out low-frequency movies that contributed to sparsity and noise. These adjustments resulted in noticeably lower prediction errors and higher top-N recommendation quality compared to baseline. Overall, this comparison highlights the strong influence of hyperparameter selection and data filtering on the effectiveness of our model.

Model	Popularity threshold	K (neighborhood size)	Rating prediction k	Recommendation k	MAE	RMSE	Precision@k	Recall@k
Baseline	40	5	5	5	0.781436	1.020824	0.212654	0.152467
Optimised	60	20	20	20	0.716065	0.929942	0.153873	0.460133

Figure 3. Baseline Model vs Optimised Model

3.0.6 *Evaluation Metrics and Results*

Our model was evaluated using both rating-prediction and top-n recommendation metrics. Across the test set, it achieved an MAE of around 0.7161 and an RMSE of approximately 0.9299, meaning the the predicted ratings were typically within one point of the actual ratings. Since these values decreased, it shows that the model is now more accurate. Figure 4 visualises that the system also reached Precision@20 of 0.1539 and Recall@20 of 0.4601. It indicates that about 15 percent of the recommended items were relevant to users and the model successfully retrieved about 46 percent of the movies each user actually liked.

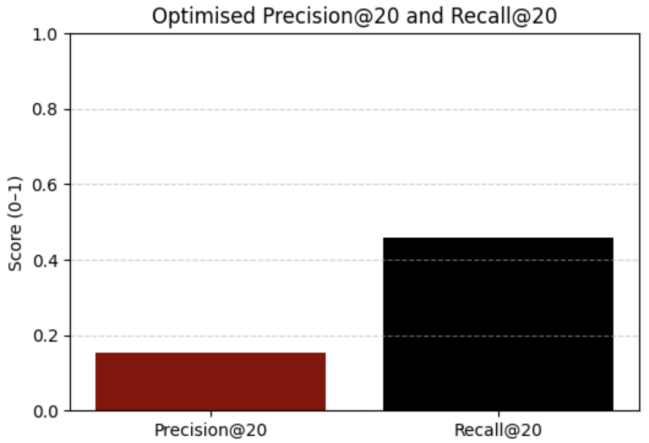


Figure 4.

4 Discussion

The optimised model performed noticeably better than baseline. Adjusting the hyperparameters reduced sparsity and resulted in more accurate and relevant predictions, showing that choosing appropriate hyperparameters is essential for improving memory-based CF. KNN remains simple and effective for personalised recommendations, but it still has limitations: it struggles with sparsity, large datasets and cold-start problem.

A movie recommendation system also raises several important considerations. Legally, any real deployment must follow data protection laws and dataset licensing rules. Socially, recommendation systems can unintentionally create filter bubbles or reinforce popularity bias, which may limit diversity. Ethically, the system should remain fair, transparent and avoid overly influencing or restricting what users see. On the professional side, developers must report model performance honestly, protect user data, and take responsibility for how recommendations influence user behaviour.

5 Conclusion and future work

Overall, the project highlights a practical method for developing a personalised recommendation system. For future work, the system could be extended with item-based KNN, different similarity methods, or additional movie information to create hybrid recommender.

ACKNOWLEDGEMENTS

I would like to thank Mohammad Majid, my module leader, Aniket Basu, my lab tutor, and the rest of teaching staff for their guidance and support during the sessions as well as my groupmates for their hard work alongside me throughout this project. I would also like to thank the creators of the MovieLens dataset for providing the data used in this project.

REFERENCES

[1] K-nearest neighbor(knn) algorithm, aug 2025.
[2] Anas Ahmed, Eslam Mohammed, Mariam Barakat, Yara Mahfouz, Youssef Hisham, and Ziad Elkady. Movie recommendation system. Math-301s Project Report, 2020. Submitted to Dr. Marwa Sorour.
[3] Kouayep Sonia Carole, Tagne Poupi Theodore Armand, and Hee Cheol Kim, 'Enhanced experiences: Benefits of ai-powered recommendation systems', in *2024 26th International Conference on Advanced Communications Technology (ICACT)*, pp. 216–220, (2024).
[4] GroupLens, 'Movielens latest datasets', (2015).
[5] Sarika Jain, Anjali Grover, Praveen Singh Thakur, and Sourabh Kumar Choudhary, 'Trends, problems and solutions of recommender system', in *International Conference on Computing, Communication Automation*, pp. 955–958, (2015).
[6] Gireen Naidu, Tranos Zuva, and Elias Mmbongeni Sibanda, 'A review of evaluation metrics in machine learning algorithms', in *Artificial Intelligence Application in Networks and Systems*, eds., Radek Silhavy and Petr Silhavy, pp. 15–25, Cham, (2023). Springer International Publishing.
[7] Carlos Pinela. Recommender systems — user-based and item-based collaborative filtering, November 2017. Accessed: 2025-11-28.
[8] Mandeep Singh, Prafull Mishra, Nitin Aggarwal, Vinay Kumar, Pramod Kumar Sharma, and Ritesh Yadav, 'Cosinerecom: A knn-based movie recommendation system using cosine similarity', in *2024 4th International Conference on Advancement in Electronics Communication Engineering (AECE)*, p. 192, (2024).
[9] Ramni Harbir Singh, Sargam Maurya, Tanisha Tripathi, Tushar Narula, and Gaurav Srivastav, 'Movie recommendation system using cosine similarity and knn', *International Journal of Engineering and Advanced Technology (IJEAT)*, **9**(5), 4, (June 2020).
[10] Yuetong Wu, 'Movie recommendation system using knn, cosine similarity and collaborative filtering', *Highlights in Science, Engineering and Technology*, (2024).